DevOps for Data Science

(All content from Gold (2024))

Rohan Alexander

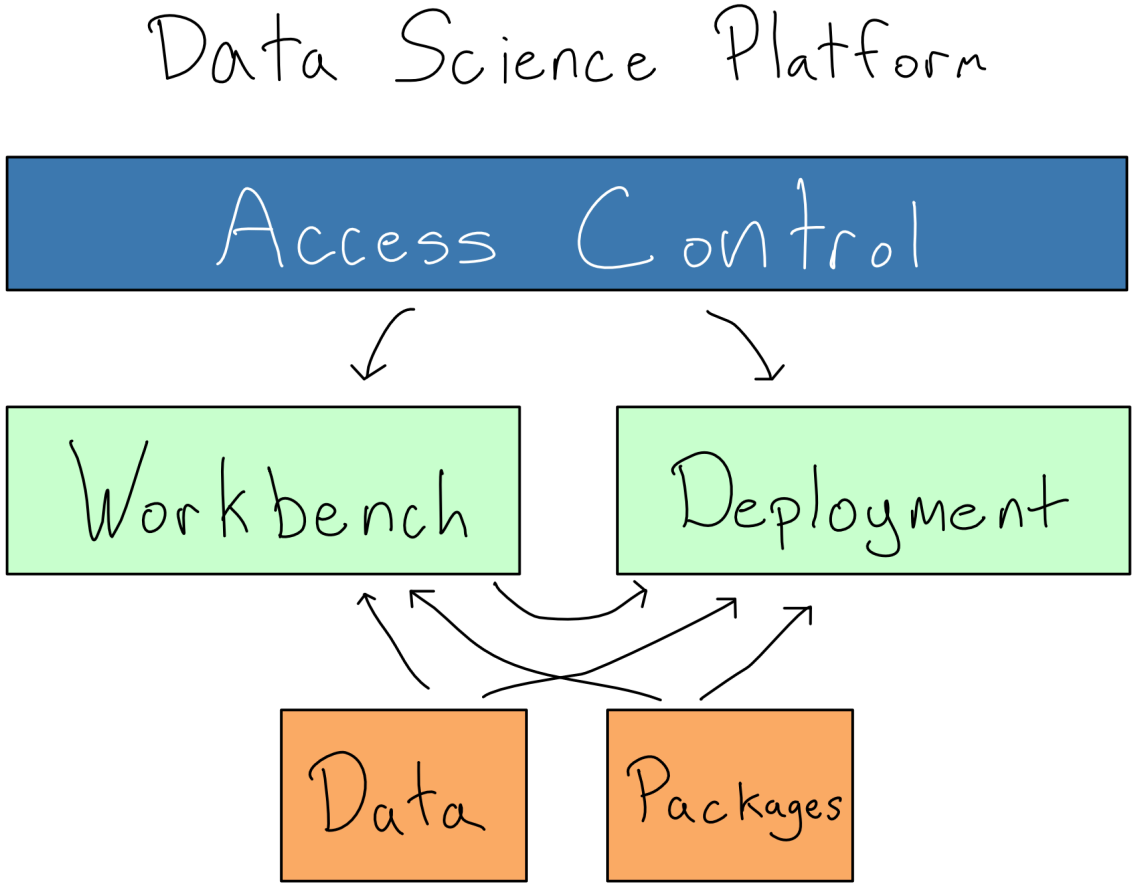
21 October, 2026

Why reproducible, portable environments matter

DevOps

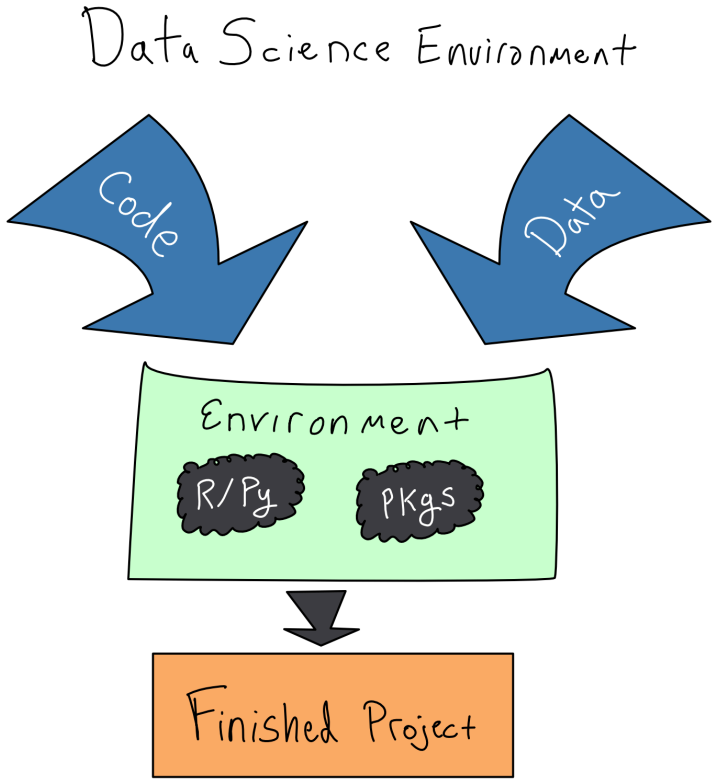
- DevOps is a set of cultural norms, practices, and tooling to help make developing and deploying software smoother and lower risk.

Two components



From Gold (2024), Introduction.

Managing environments



From Gold (2024), Chapter 1.

Discussion: Have you tried to share your work before and not had it run on someone else's computer?

Environments have layers

Layers of data science environments

Layer	Contents
Packages	Python + R Packages
System	Python + R Language Versions
	Other System Libraries
	Operating System
Hardware	Virtual Hardware
	Physical Hardware

From Gold (2024), Chapter 1.

The package layer

There are three different places packages can live:

1. In a repository.
2. In a library.
3. Loaded.

Success means:

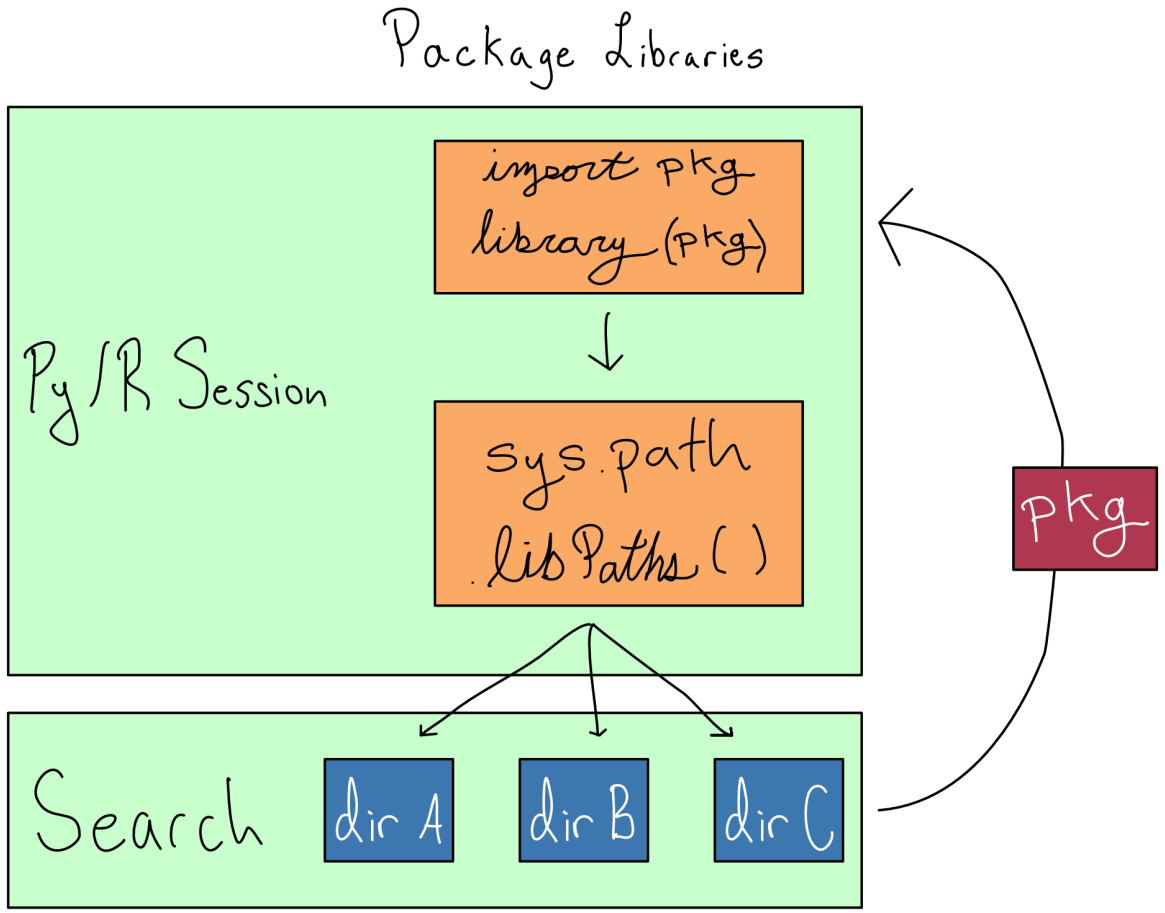
1. Isolation.
2. Transportability.

Using virtual environments

1. Create standalone package libraries
2. Document environment state
3. Collaborate or deploy
4. Use a virtual environment

(We use uv for this, cf Gold (2024) who uses venv.)

What's happening under the hood



From Gold (2024), Chapter 1 "An import or library statement triggers a call to `sys.path`, which returns directories to search. That returns a package to the session".

Data project architecture

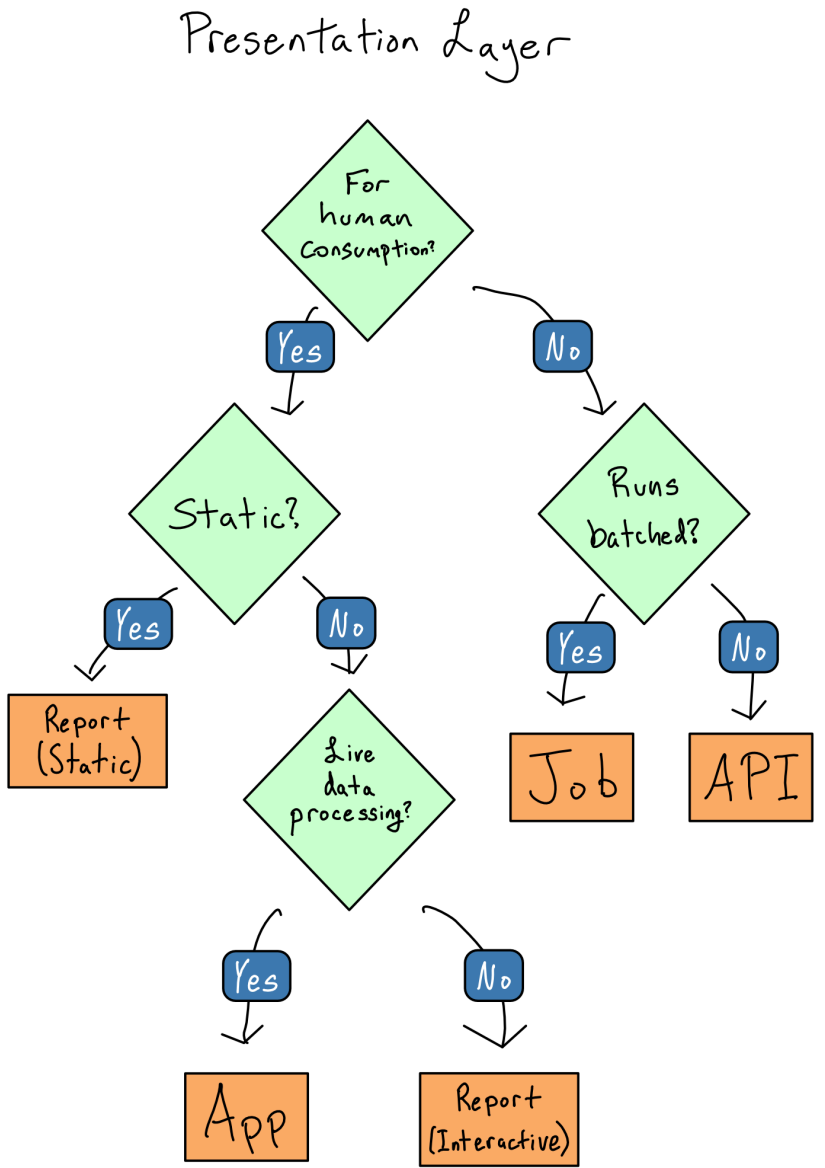
Front-end vs back-end

- Normal software engineering approach:
  1. Presentation layer
  2. Processing layer
  3. Data layer
- Doesn't exactly map to data science, but we can still benefit from thinking seriously about it, especially the presentation layer.

Discussion: Have you thought about separating aspects in this way before?

Presentation layer

1. A job.
2. An app.
3. A report.
4. An API.



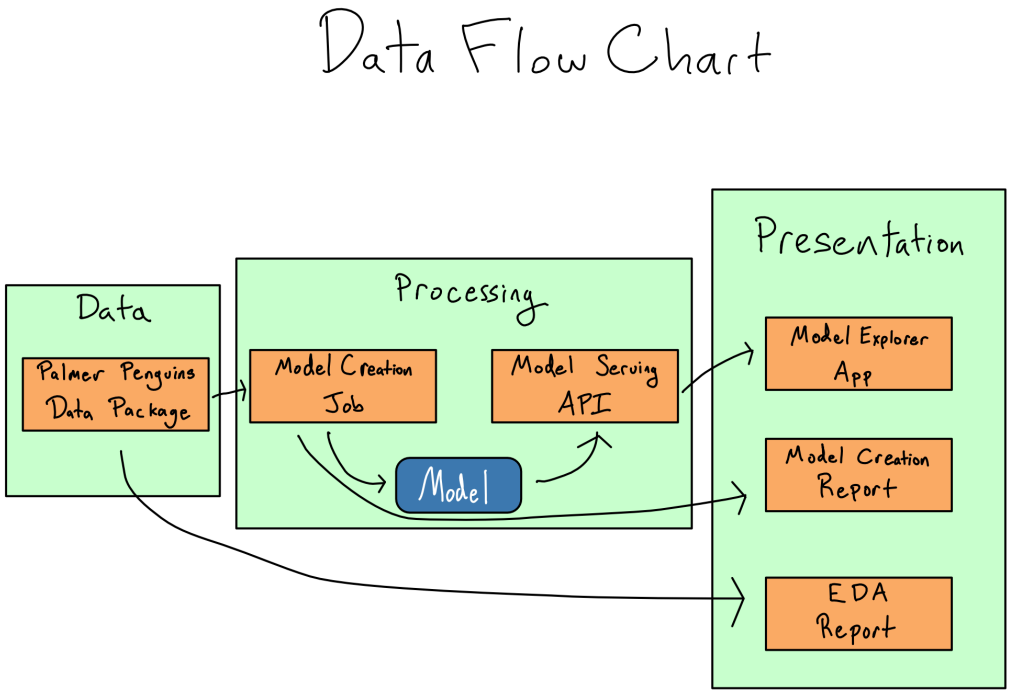
Do less in the presentation layer

- In `scripts` create `analysis_data.parquet`, which is then loaded in `paper.qmd`.
- There should be no (or at least minimal) manipulation of `analysis_data.parquet` required within `paper.qmd`.

Smaller data in the presentation layer (initially)

- Everything is easier when your data is small.

Write a data flow chart



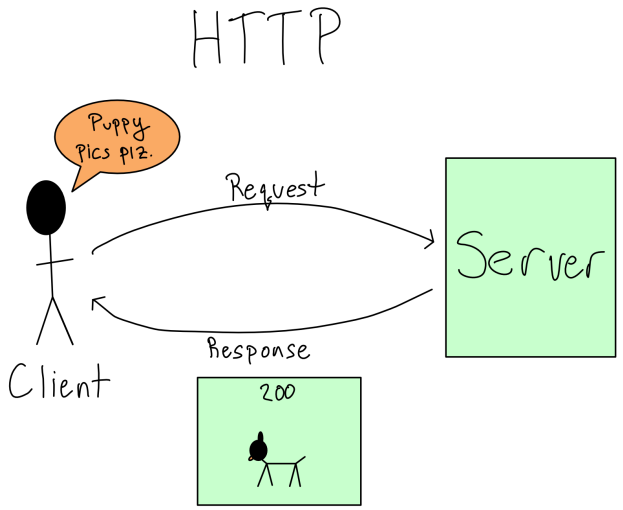
From Gold (2024), Chapter 2.

Activity: In your groups, with pen and paper please write out your data flow chart.

Secure data access

Connecting to APIs

- APIs are the standard way for two computer systems to communicate.



From Gold (2024), Chapter 3.

Discussion: Which APIs have you used?

API endpoints and paths and HTTP verbs

- Each request to an API is directed to a specific endpoint.
- The HTTP verb, also known as the request method, describes the type of operation you’re asking for.
- Commonly used verbs are: GET and POST.

Request parameters and bodies and (Auth) headers

- Each endpoint accepts specific arguments in a required format
- Most APIs require authentication. The most common forms of authentication are a username and password combination, an API key, or an OAuth token.
- The contents of the response are in the body.

Request Status Codes

- 200: a successful response.
- 3xx: Your query was redirected somewhere else.
- 4xx: Errors with the request.
- 400: Bad request. This isn't a request the server can understand.
- 401/403: Unauthorized or forbidden. Required authentication hasn't been provided.
- 404: Not found. There isn't any content to access here.

Activity: Please partner with someone new and use `requests` to get the NASA APOD using their API: https://api.nasa.gov/.

Environment variables

- Never put credentials, such as username, password, or API key, in code.
- The power of using an environment variable is that you reference them by name.
- `os.getenv("<VAR_NAME>")`

Activity: What is the name of the environment variable that stores my OpenAI key in https://zenodo.org/records/17106979? (`/scripts/get_codes.py`)

Setting environment variables

Development environment:

- Read a `.env` file into your session, likely using `python-dotenv` or `os.getenv()`.

Production environment:

- More difficult. GitHub Actions allows you to set SECRETS.

Logging and monitoring

Observability

Two ways of being wrong:

1. You’re wrong and you know it.
2. You’re wrong and you don’t know it.

When it comes to data science if we’re wrong we want to know about it. Observability is the way to enable this. Two aspects:

1. Emitting the logs and metrics.
2. Aggregating and consuming them.

Observing correctness

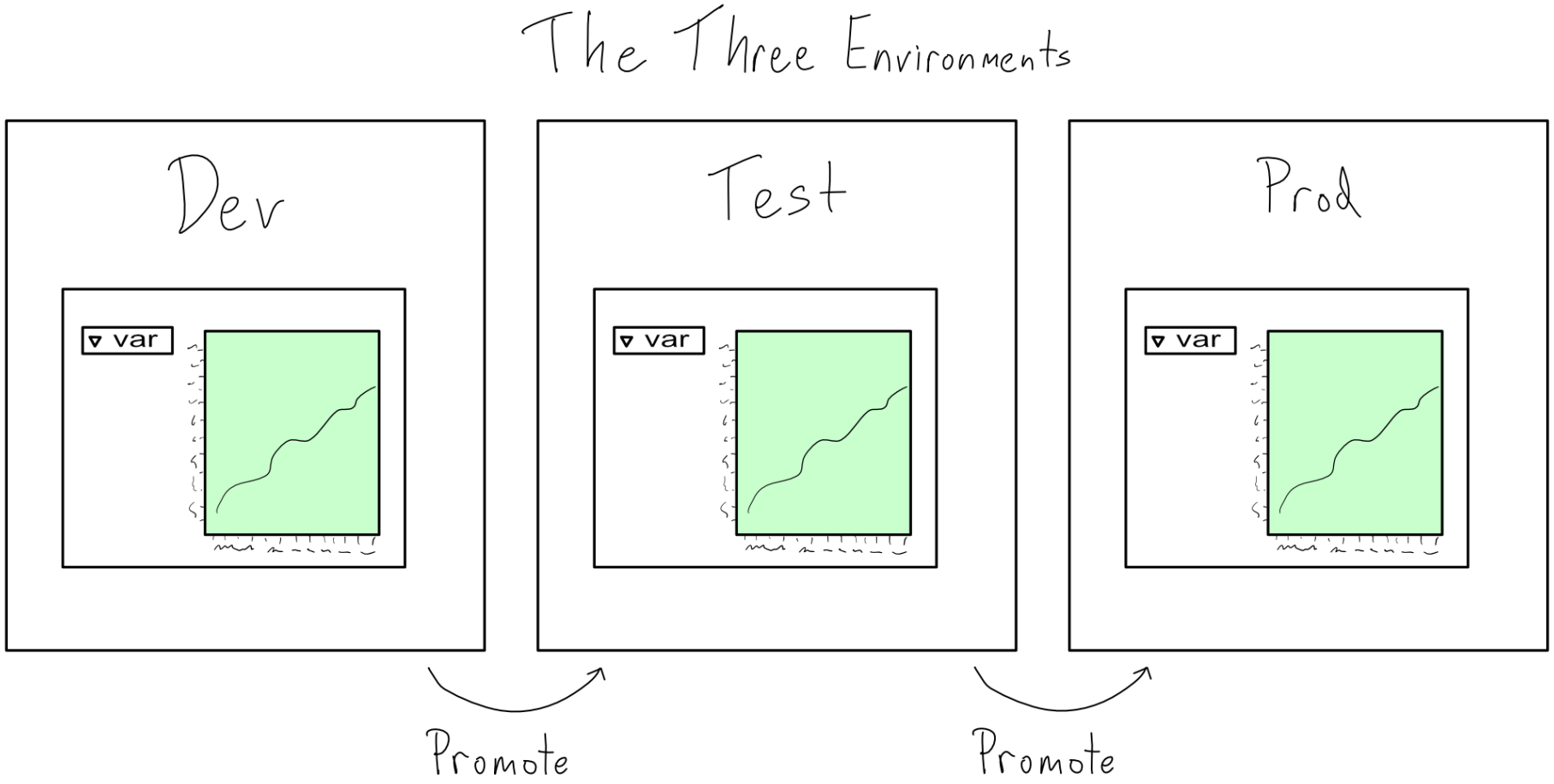
- An issue that doesn’t result in code failure but yields incorrect answers is especially concerning e.g. data joins usually complete even if the merge quality is terrible. Model APIs will return a prediction even if the prediction is bad.
- To address this we use process metrics and simulation.
- Test: joins, cross-tabs, goodness-of-fit metrics, data types.

Deployments and code promotion

CI/CD

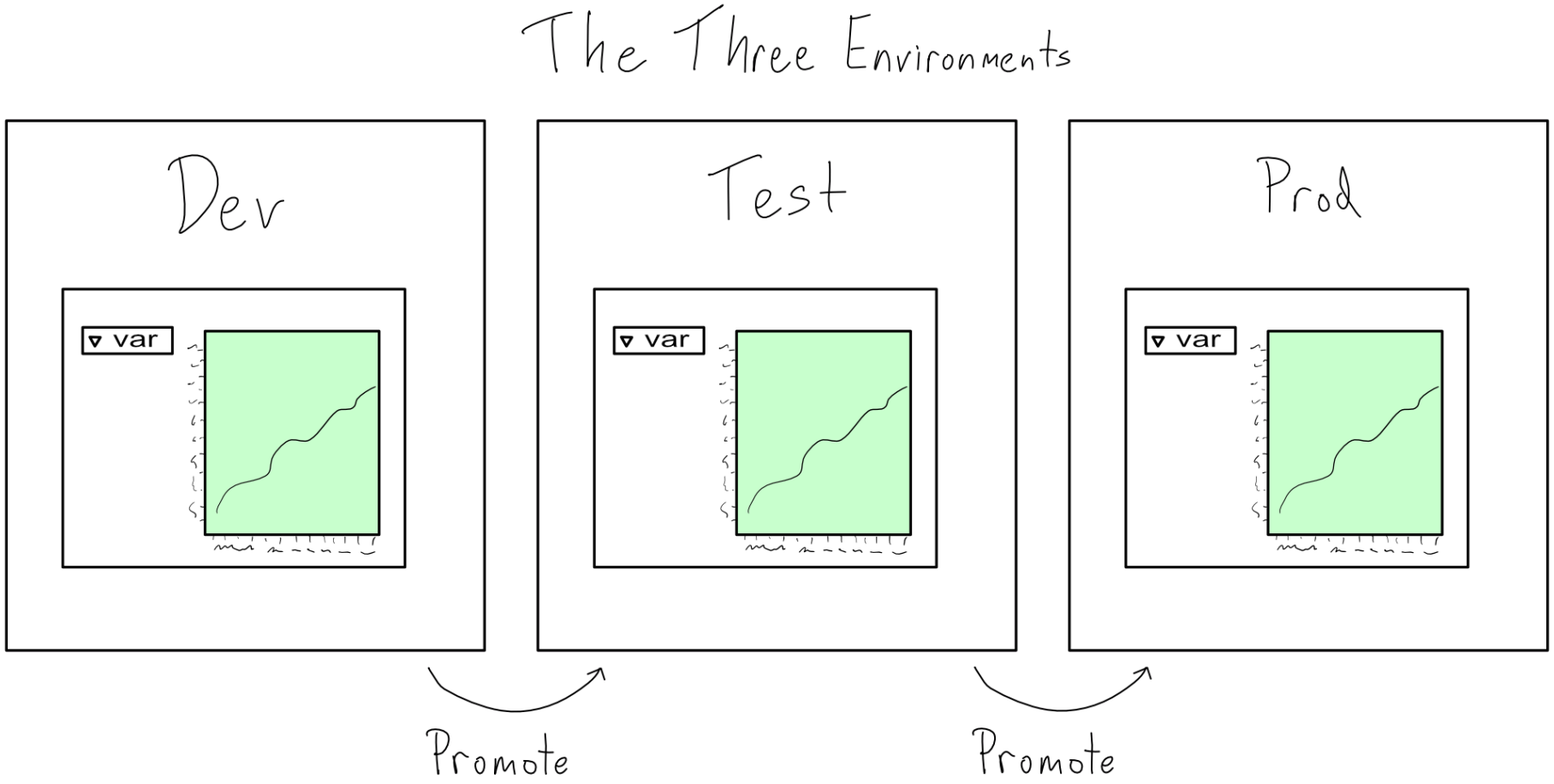
- CI/CD, which is short for Continuous Integration and Continuous Deployment and Continuous Delivery.
- A few of the principles of CI/CD workflows include:
    1. Central availability of source code.
    2. Frequent and incremental additions.
    3. Automation of deployment.
    4. Automated testing.

Separate the prod environment



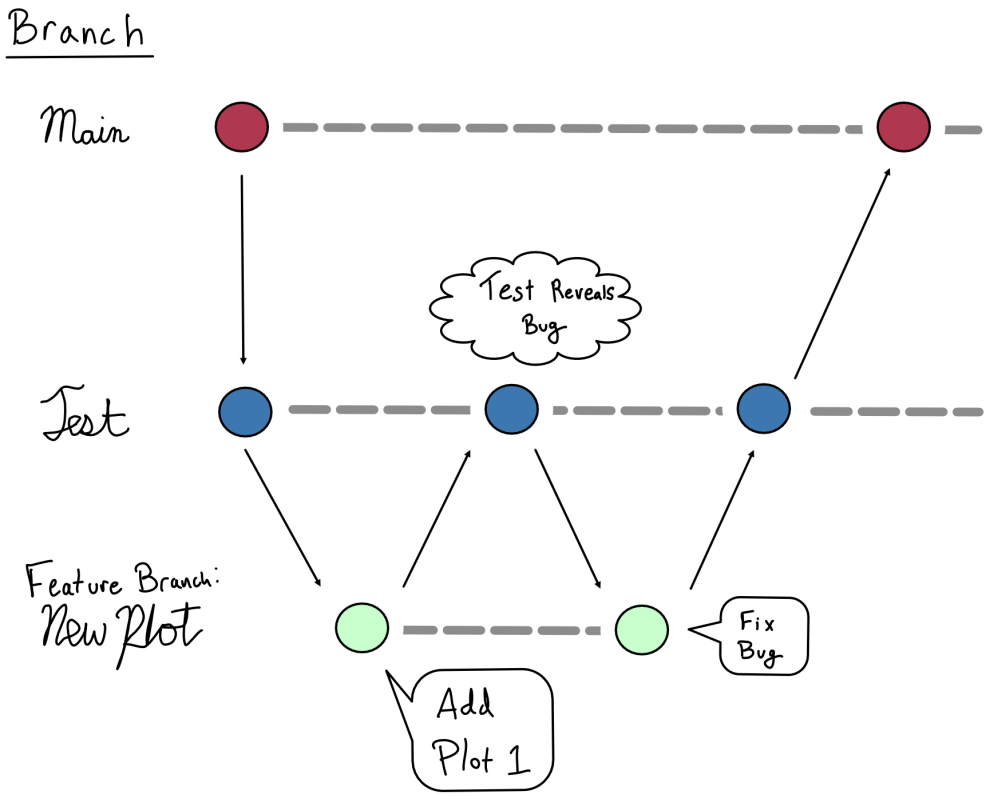
Gold (2024), Chapter 5.

Dev and test environments



Gold (2024), Chapter 5.

Version control implements code promotion



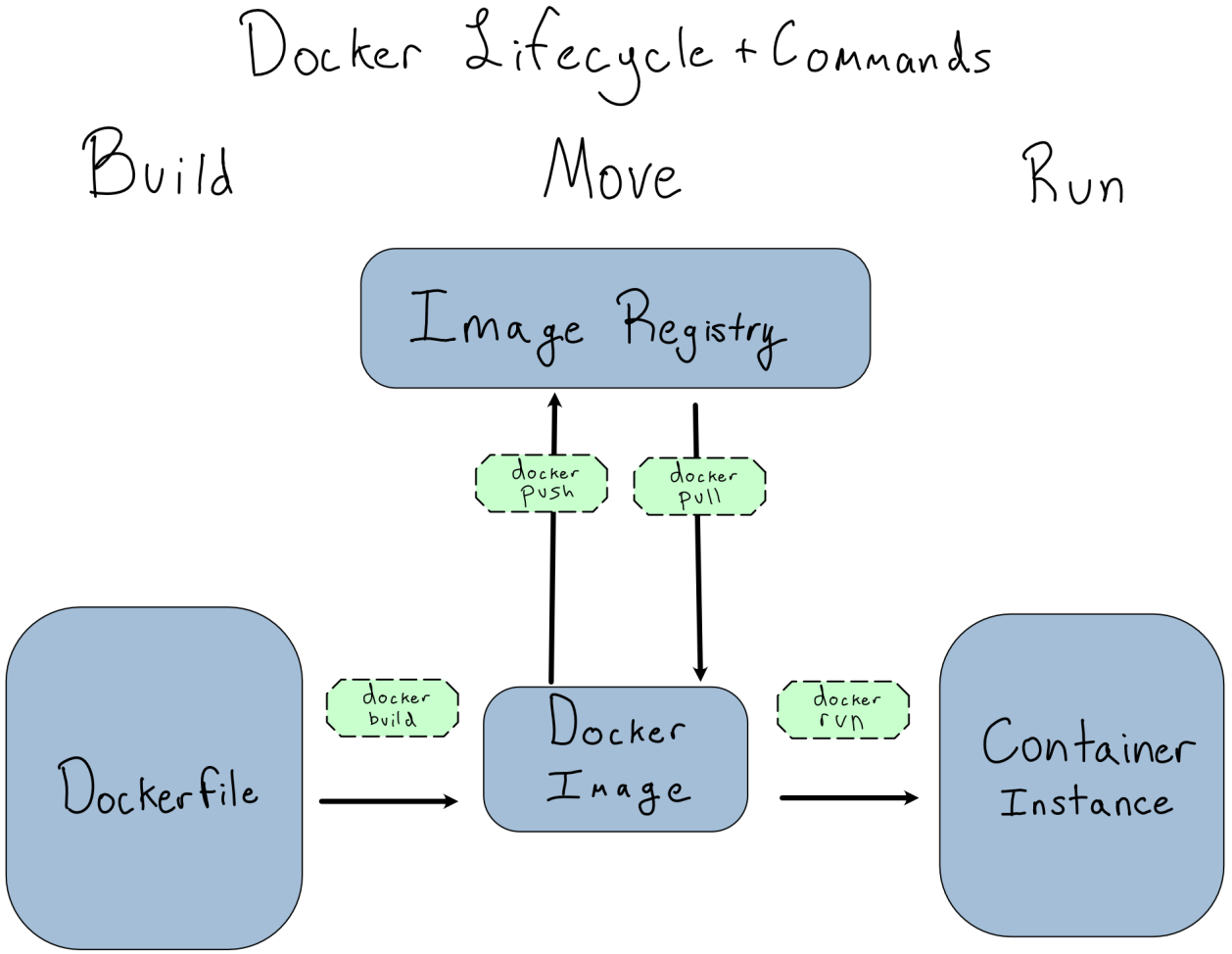
Gold (2024), Chapter 5.

Docker

Overview

- Docker is an open-source tool for building, sharing, and running software.
- Earlier we talked about ensuring others could reproduce our environment; Docker's approach is "why don't we just give them the environment".

Container lifecycle



Gold (2024), Chapter 6.

Running containers

- You must know which image you’re referencing to build, push, pull, or run it.
- The docker run command runs container images as an instance.

Building images from Dockerfiles

- A Dockerfile is a set of instructions to build a Docker image.
  - FROM – Specify the base image, usually the first line of the Dockerfile.
  - RUN – Run any command as if you were sitting at the command line inside the container.
  - COPY – Copy a file from the host filesystem into the container.
  - CMD – Specify what command to run on the container's shell when it runs, usually the last line of the Dockerfile.

References

Gold, Alex K. 2024. *DevOps for Data Science*.